

High Performance Computing

Unit: v

OpenMP

(B Tech 2nd Year)

Mr. Tushar
Assistant Professor
Department of CS

FACULTY PROFILE

Name of Faculty: Mr. Tushar

Designation & Department: Assistant Professor, CS

Qualification: B.E, M.Tech

Experience: 1 Year

- B.Tech. (Computer Science AIML)
- Core subject
- 3rd SEMESTER

PREDICTIVE ANALYTICS

L	T	P	C
3	0	0	3

Course Contents / Syllabus

Course Contents / Syllabus		
Module 1	Introduction to HPC	10 hours
Computational Thinking, Computing, Why High-Performance Computing, Applications of High-Performance Computing, Parallel programming Software Platforms and its significance, Cloud computing, Grid computing, Cluster computing and Quantum Computing Multi-core CPUs, Graphical Processing Units.		
Module 2	Computing Architectures	10 hours
SISD, SIMD, MISD, MIMD, Memory Hierarchy, Data parallelism, Task parallelism, Bit-level parallelism, Instruction-level parallelism Concurrency, Decomposition, Mapping, PRAM, NUMA, Multithreading vs Multiprocessing, shared memory model, Open MP distributed memory model, Message passing interface. Performance measures: Speedup, efficiency and scalability.		
Module 3	Distributed Memory	10 hours
Distributed memory and Message passing networks, Broadcast, Reduction, Parallel Prefix Sum, Scatter, Gather, Network topologies for parallel computing, Network optimization. Distributed BFS, Graphs and adjacency matrix, Matrix based BFS, CUDA programming Parallel matrix operations, Sparse vs Dense matrices.		
Module 4	Cluster Computing	9 hours
BLAS, LAPAC, Cluster Computing, Clustering Models, Clustering Architectures, Clustering Architectures key factors, types of clusters, Mission critical Vs Business Critical Applications, Fault Detection and Masking Algorithms, Check pointing, Heartbeats Watchdog Timers, Fault recovery through Failover and Failback Concepts.		
Module 5	OpenMP	9 hours
OpenMP, Key concepts, Fork-join model, Goals of OpenMP, <u>Supported</u> platforms and compilers, OpenMP API components: directives runtime routines, environment variables, General code structure, Core syntax and structured blocks, Compiling OpenMP programs Parallel region construct, Creating and managing threads, Specifying the number of threads, Thread management and thread IDs Applications of OpenMP.		
Total Lecture Hours		48 hours

Book Title/ External Links as per Syllabus

<u>S.No</u>	Book Title	Author
1	The Sourcebook of Parallel Computing	Jack Dongarra, Geoffrey Fox, and Ken Kennedy
2	<u>Petascale</u> Computing: Algorithms and Applications	David A. Bader (Ed.)
3	Parallel and <u>High Performance</u> Computing	Robert Robey and Yuliana Zamora
4	High Performance Computing: Modern Systems and Practices	Thomas Sterling, Maciej Brodowicz, and Matthew Anderson

Reference Books:

<u>S.No</u>	Book Title	Author
1	High Performance Computing: Modern Systems and Practices	Thomas Sterling, Matthew Anderson, and Maciej Brodowicz
2	Using OpenMP: Portable Shared Memory Parallel Programming	Barbara Chapman, Gabriele Jost, and Ruud van der Pas
3	Introduction to Parallel Computing	Ananth Grama, Anshul Gupta, George Karypis, and Vipin Kumar
4	The OpenMP Common Core: A Hands-on Exploration	Tim Mattson et al.
5	A Hands-on Introduction to OpenMP	Tim Mattson and Larry Meadows
6	OpenMP Tasking Explained	Ruud van der Pas
7	An Introduction to Parallel Programming	Peter Pacheco

NPTEL/ Youtube/ Faculty Video Link:

Unit 1	https://youtu.be/tGIobcyKViI?si=e6QKYtz8z1KHG5GI
Unit 2	https://www.youtube.com/watch?v=FTXikoQr46U
Unit 3	https://www.youtube.com/watch?v=gE8KgD9D69k
Unit 4	https://youtu.be/7rooWbLe1iI?si=hgkzh6bYDcxLxRQd
Unit 5	https://youtu.be/gcadMpUXcXs?si=DCWMph8Hgn6bt6-g

Branch wise Applications

- Computer science encompasses a diverse array of specialized fields that address modern technological challenges and innovations. **Artificial Intelligence (AI) & Machine Learning (ML)** empower systems to perceive, learn, and make decisions—from voice assistants and medical diagnostic tools to autonomous vehicles . **Data Science & Big Data** blend statistics, data modeling, and programming to uncover insights for business intelligence, fraud detection, and predictive analytics. **Software Engineering** focuses on the full lifecycle of building robust applications—from mobile and web platforms to enterprise and industrial systems . In **Cybersecurity & Cryptography**, specialists design secure protocols and systems to protect networks, financial infrastructure, and sensitive personal data. **Cloud Computing & DevOps** enable scalable on-demand services, powering online gaming, streaming, and enterprise applications with automated deployment pipelines . Fields like **Computer Graphics & Visualization**, **Computer Vision**, and **HCI** focus on visual, interactive, and user-centric computing, driving VR/AR, gaming, medical imaging, and intuitive interfaces. **Algorithms & Data Structures** form the theoretical backbone, optimizing search, encryption, and data retrieval while informing compiler and database design. At the hardware level, **Computer Architecture & Embedded Systems** support IoT, smart devices, and robotics through efficient design of processors and sensors . **Blockchain & Distributed Systems** revolutionize secure, decentralized applications in finance and supply chains. Emerging areas like **Quantum Computing** and **Neuromorphic/Biological Computing** promise breakthroughs in simulation, optimization, and ultra-efficient intelligent systems—whether using qubits to accelerate drug discovery or neuronal circuits to mimic brain-like computation .

The objective of this course is -

- **Fundamental HPC Concepts & Terminology** Introduce students to essential HPC concepts like parallelism, scaling, and performance (speedup vs. efficiency). Explain why HPC is vital across scientific and engineering domains
- **Understanding Modern Architectures** Cover shared-memory and distributed systems (clusters, GPUs, accelerators), their architectures, and how hardware design influences software performance.
- **Mastery of Parallel Programming Models** Teach key parallel programming paradigms:
 - Shared memory: OpenMP
 - Distributed memory: MPI
 - Hardware-accelerated: CUDA, OpenCL, OpenACC

Course Outcome: After completion of the course, the student will be able to		Bloom's Knowledge Level (KL)
CO1	Identify the need, applications of HPC and emerging paradigms.	K2
CO2	Analyze architectures and parallelism for HPC performance.	K4
CO3	Apply message passing for parallel algorithm design.	K3
CO4	Analyze fault tolerance in cluster computing.	K4
CO5	Implement shared memory programs using OpenMP.	K3

Engineering Graduates will be able to Understand:

1. Engineering knowledge
2. Problem analysis
3. Design/development of solutions
4. Conduct investigations of complex
5. Modern tool usage
6. The engineer and society
7. Environment and sustainability
8. Ethics
9. Individual and team work
10. Communication
11. Project management and finance
12. Life-long learning

CO-PO Mapping

CO-PO Mapping (Scale 1: Low, 2: Medium, 3: High)

CO-PO Mapping	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
C01	3	2	1	1	2	1	1	1	-	2	1	2	3	3
C02	3	3	2	2	3	-	-	-	1	1	1	3	3	3
C03	3	3	3	2	3	-	-	-	2	1	1	3	3	3
C04	2	3	2	2	2	-	-	1	2	1	1	3	3	3
C05	2	2	3	2	3	-	-	-	1	1	1	3	3	3

PSO's

At the end of the program, the student will be able to-

PSO 1: Apply suitable techniques, modern tools to solve complex engineering problems in the field of computer science.

PSO 2: Design and optimize, ethical and innovative solutions for the betterment of the society.

PSO 3: Work as an individual or lead in a team with good communication and engage in life-long learning.

COs and PSOs Mapping

Course Outcome	PSO1	PSO2	PSO3
CO1	3	2	–
CO2	3	3	–
CO3	2	3	–
CO4	2	3	–
CO5	1	3	–
Average	2.2	2.8	–

PEO's

The graduates of B.Tech Computer Science program will-

PEO1: Have successful careers in the field of Computer Science and pursue higher education.

PEO2: Engage in research and provide innovative solution to real world problems of the society with ethical responsibilities.

PEO3: Demonstrate effective communication, work as an individual or as a team and engage in life-long learning.

Question Paper Template

Printed page:

Subject Code:

Roll No:

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY ,GREATER NOIDA

(An Autonomous Institute Affiliated to AKTU, Lucknow)

B.Tech/B.Voc./MBA/MCA/M.Tech (Integrated)

(SEM: THEORY EXAMINATION (2020-2021))

Subject

Time: 3 Hours

Max. Marks:100

General Instructions:

- All questions are compulsory. Answers should be brief and to the point.
- This Question paper consists ofpages & ...8.....questions.
- It comprises of three Sections, A, B, and C. You are to attempt all the sections.
- **Section A** -Question No- 1 is objective type questions carrying 1 mark each, Question No- 2 is very short answer type carrying 2 mark each. You are expected to answer them as directed.
- **Section B** - Question No-3 is Long answer type -I questions with external choice carrying 6 marks each. You need to attempt any five out of seven questions given.
- **Section C** - Question No. 4-8 are Long answer type -II (within unit choice) questions carrying 10 marks each. You need to attempt any one part a or b.
- Students are instructed to cross the blank sheets before handing over the answer sheet to the invigilator.
- No sheet should be left blank. Any written material after a blank sheet will not be evaluated/checked.

Question Paper Template

SECTION – A			CO
1.	Attempt all parts-	[10×1=10]	
1-a.	Question-	(1)	
1-b.	Question-	(1)	
1-c.	Question-	(1)	
1-d.	Question-	(1)	
1-e.	Question-	(1)	
1-f.	Question-	(1)	
1-g.	Question-	(1)	
1-h.	Question-	(1)	
1-i.	Question-	(1)	
1-j.	Question-	(1)	
2.	Attempt all parts-	[5×2=10]	CO
2-a.	Question-	(2)	
2-b.	Question-	(2)	
2-c.	Question-	(2)	
2-d.	Question-	(2)	
2-e.	Question-	(2)	
SECTION – B			CO
3.	Answer any <u>five</u> of the following-	[5×6=30]	
3-a.	Question-	(6)	
3-b.	Question-	(6)	
3-c.	Question-	(6)	
3-d.	Question-	(6)	
3-e.	Question-	(6)	
3-f.	Question-	(6)	
3-g.	Question-	(6)	
SECTION – C			CO
4.	Answer any <u>one</u> of the following-	[5×10=50]	
4-a.	Question-	(10)	
4-b.	Question-	(10)	
5.	Answer any one of the following-		
5-a.	Question-	(10)	
5-b.	Question-	(10)	
6.	Answer any one of the following-		
6-a.	Question-	(10)	
6-b.	Question-	(10)	
7.	Answer any one of the following-		
7-a.	Question-	(10)	
7-b.	Question-	(10)	
8.	Answer any one of the following-		
8-a.	Question-	(10)	
8-b.	Question-	(10)	

Subject Result: NA

Department Result: NA

Faculty-Wise Result: NA

Prerequisites:

- **Programming Fundamentals:**

Languages: Proficiency in C, C++, or Fortran – essential for working

Recap:

- High-Performance Computing, Applications of High-Performance Computing

High Performance Computing (HPC) focuses on solving complex computational problems using parallel and distributed computing technologies. In today's era of large-scale data and simulation-driven systems, HPC enables faster processing and analysis by harnessing the power of modern hardware and software tools like multi-core processors, clusters, grids, and even quantum systems.

Students learn:

- Core concepts of parallelism
- Architectural models like SIMD, MIMD
- Programming using OpenMP and MPI
- Fault-tolerant mechanisms in cluster systems

This course builds the foundation to design, implement, and analyze performance-driven software and systems used in scientific research, machine learning, big data, etc.

What is OpenMP?

- Open Multi-Processing API for shared memory parallel programming

Developed for:

- C, C++, Fortran

Used for:

- Multi-core processors
- Parallelizing loops, tasks, and regions of code

- OpenMP (Open Multi-Processing) is an API that supports multi-platform shared memory multiprocessing programming in C, C++, and Fortran.
- It allows the creation of parallel programs easily using compiler directives
- .OpenMP is portable and scalable across different architectures (Windows, Linux, UNIX).
- Mainly used for shared memory architectures.

OpenMP: Key Concepts

Concept	Description
Shared Memory	All threads can access global (shared) variables.
Parallel Region	A section of code that runs in parallel using multiple threads.
Work Sharing	Splitting tasks (like loops) among threads.
Synchronization	Controlling access to shared data to avoid race conditions.
Fork-Join Model	Master thread creates ("forks") multiple threads; at the end, they join back.

1.What does OpenMP stand for?

- A) Open Multiprocessing Protocol
- B) Open Memory Processing
- C) Open Multi-Processing
- D) Open Multi-Processing**

2.OpenMP is primarily designed for which type of memory model?

- A) Distributed memory
- B) Shared memory**
- C) Virtual memory
- D) Flash memory

3.Which programming languages is OpenMP developed for?

- A) Python, Java, and C#
- B) C++, Rust, and Go
- C) C, C++, and Fortran**
- D) JavaScript, PHP, and Perl

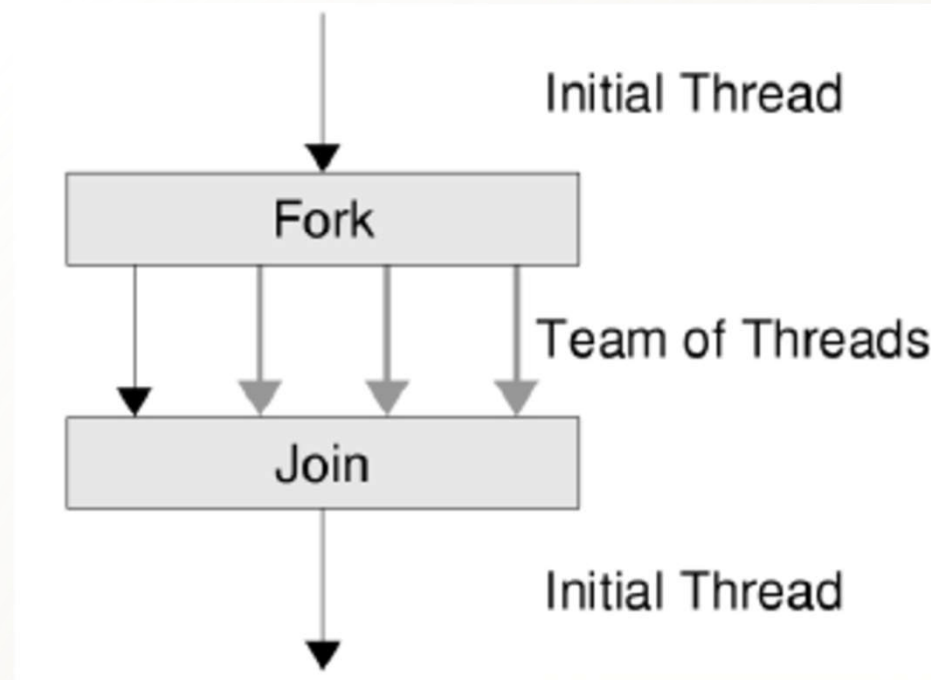
4.What is the primary use of OpenMP?

- A) Encrypting files using multiple threads
- B) Managing distributed database clusters
- C) Parallelizing loops, tasks, and regions of code**
- D) Compressing multimedia files

5.OpenMP is typically used on which type of processors?

- A) Single-core processors
- B) ARM microcontrollers
- C) Multi-core processors**
- D) GPU clusters only

Key concepts



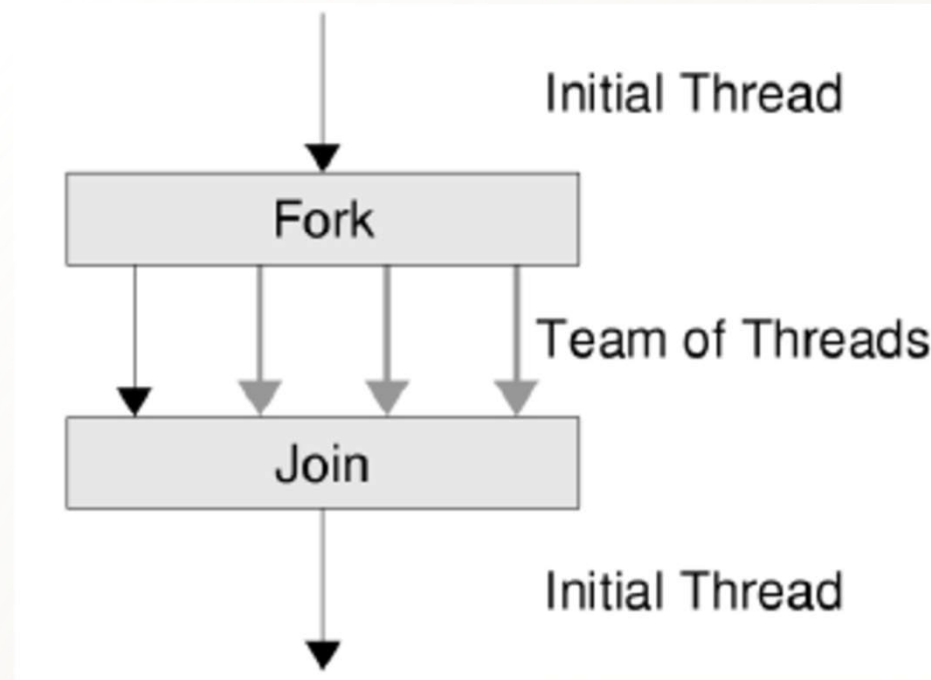
Fork-Join Model

- Fork: The master thread creates (forks) multiple threads to execute tasks in parallel.
- Join: After completing the parallel tasks, threads synchronize (join) back into the master thread.

Key concepts

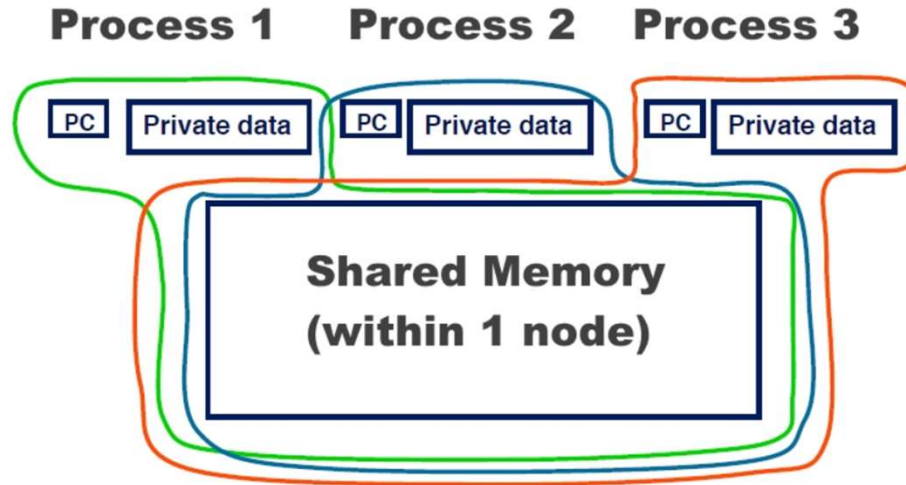
- The Fork-Join model is the core execution model used in OpenMP to enable parallelism. It begins with a single master thread, which runs the program sequentially until it encounters a parallel region.
- Fork Phase: When the program reaches a parallel block (e.g., `#pragma omp parallel`), the master thread forks a team of worker threads. All threads, including the master, then execute the code inside the parallel region concurrently. The number of threads created depends on system settings or user-defined configuration (like `omp_set_num_threads()` or `OMP_NUM_THREADS`).
- Join Phase: After the parallel region is completed, all worker threads join back with the master thread. At this point, only the master thread continues executing the remaining sequential part of the program.

Key concepts



- This model enables developers to parallelize specific sections of code while keeping the rest of the program simple and serial. It's efficient for tasks like loop parallelism and data processing, where sections of code can be executed independently across multiple threads.

Key concepts



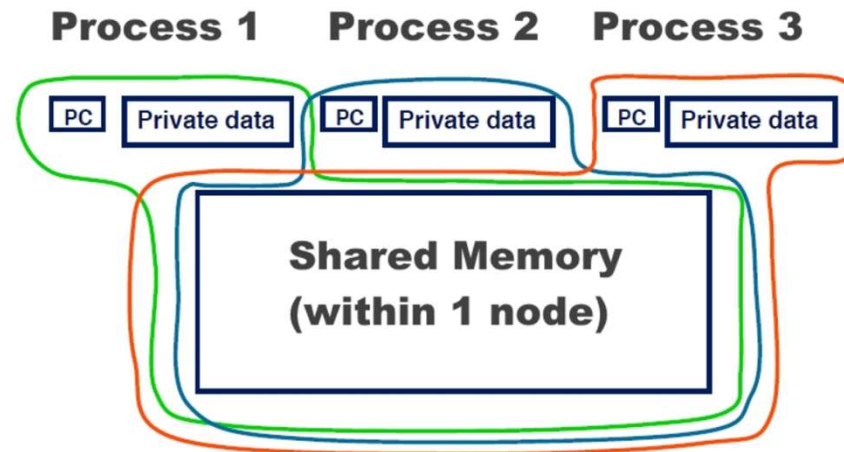
Shared vs Private Variables

- Shared: Variables accessible by all threads (e.g., global counters).
- Private: Each thread has its own copy; changes are not seen by others.

Key concepts

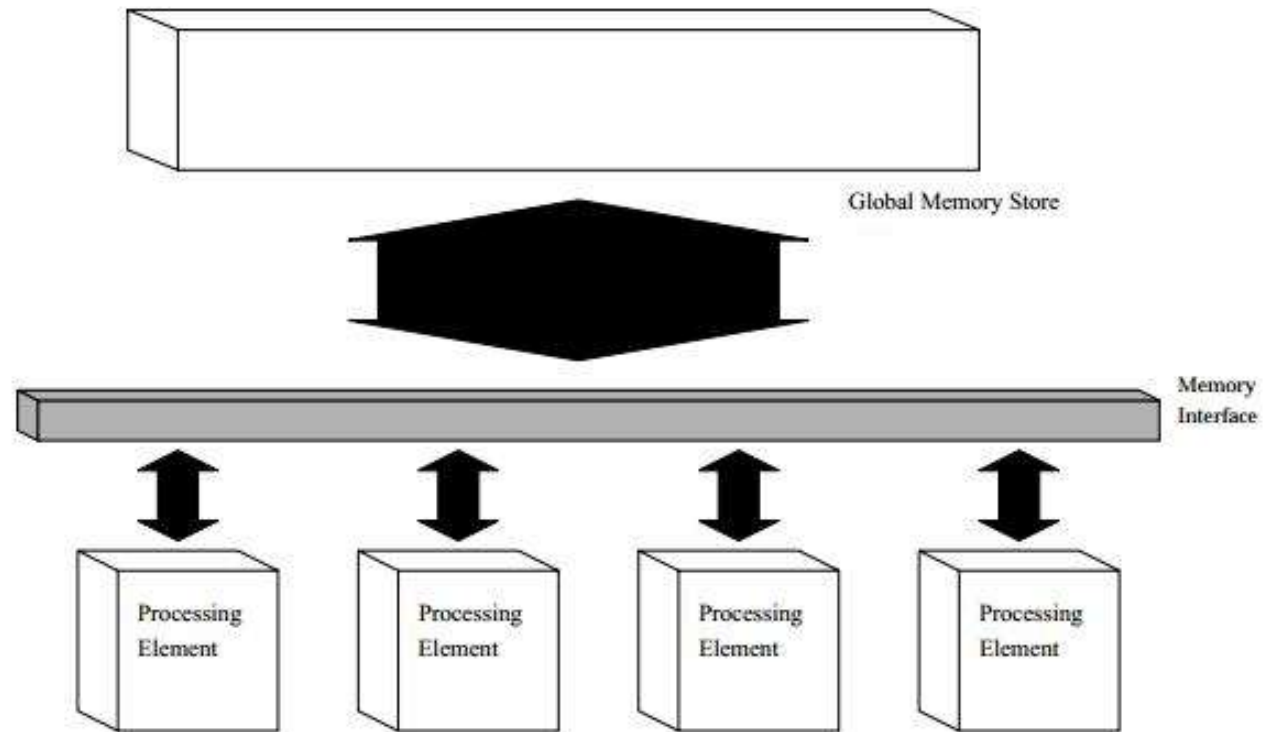
- Definition: Shared variables are those that all threads can access and modify during parallel execution.
- memory Behavior: A single memory location is used by all threads, meaning changes made by one thread are visible to all others.
- Common Use Cases: Useful when multiple threads need to update or read the same data — for example, updating a shared counter or accumulating a global sum.
- Risk: Shared variables can lead to race conditions, where the result depends on the timing of thread execution. To prevent this, synchronization techniques like `#pragma omp critical`, `#pragma omp atomic`, or reduction should be used.

Key concepts



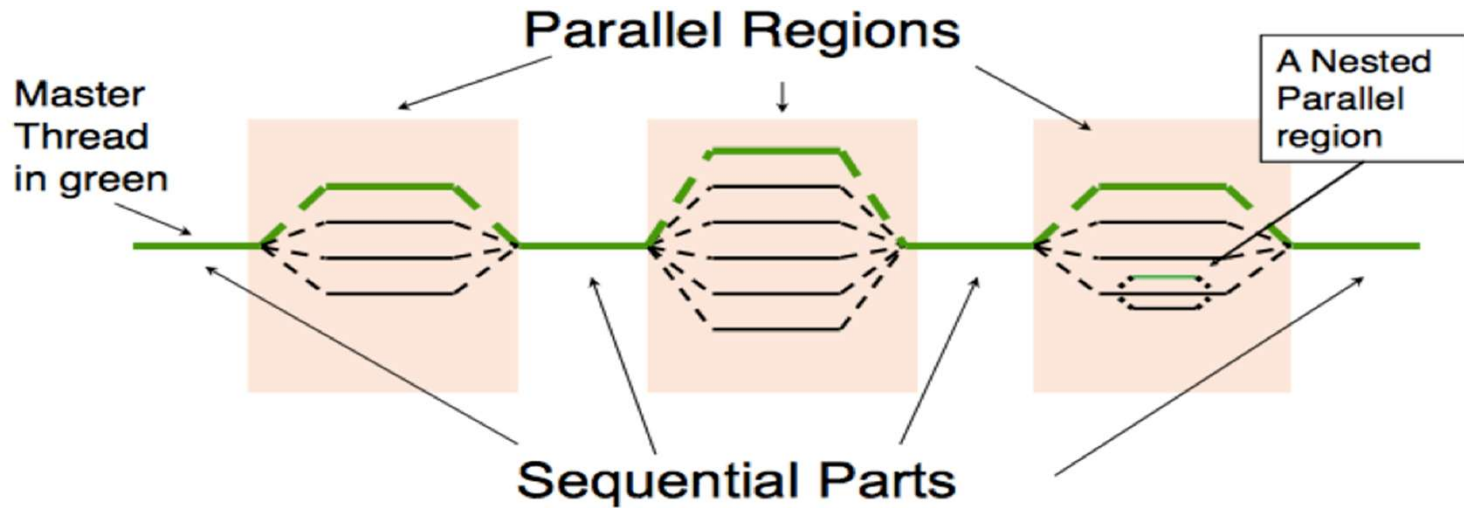
Feature	Shared	Private
Accessibility	All threads access same variable	Each thread has its own copy
Memory location	One shared location	Separate location per thread
Visibility	Changes visible to all threads	Changes visible only to that thread
Risk	Race conditions if unsynchronized	No risk of interference

Key concepts



Schematic of a shared memory architecture

Key concepts



Thread-based Parallelism

- Multiple threads run simultaneously, each handling a portion of the task.
- Improves performance on multi-core processors by dividing work among cores.

Key concepts

Thread-based Parallelism

• Thread-based parallelism is a programming model where multiple threads are created to execute tasks concurrently. Each thread performs a part of the overall task, enabling the program to utilize multiple processor cores simultaneously for faster execution.

How It Works:

- The main program creates multiple threads.
- These threads execute **in parallel**, sharing the system's CPU resources.
- The workload is **divided** among the threads so that each one handles a specific portion.
- Once the tasks are complete, threads may **synchronize** to combine results or continue with dependent operations.

Key concepts

`#pragma omp parallel for`

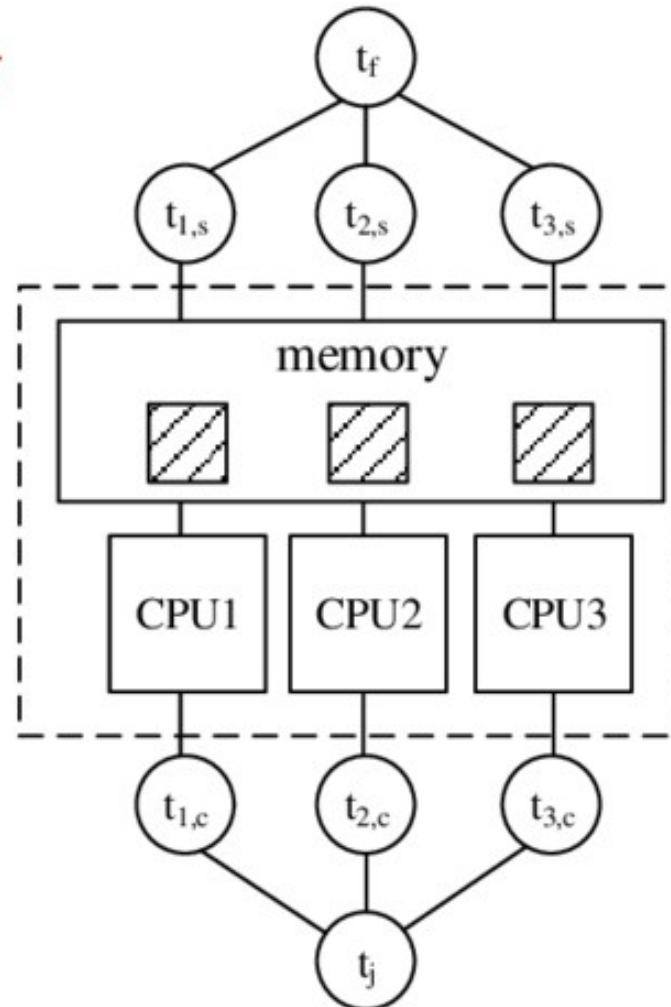
{
 Start
 (e.g. for each node)

structured
parallel block

⋮

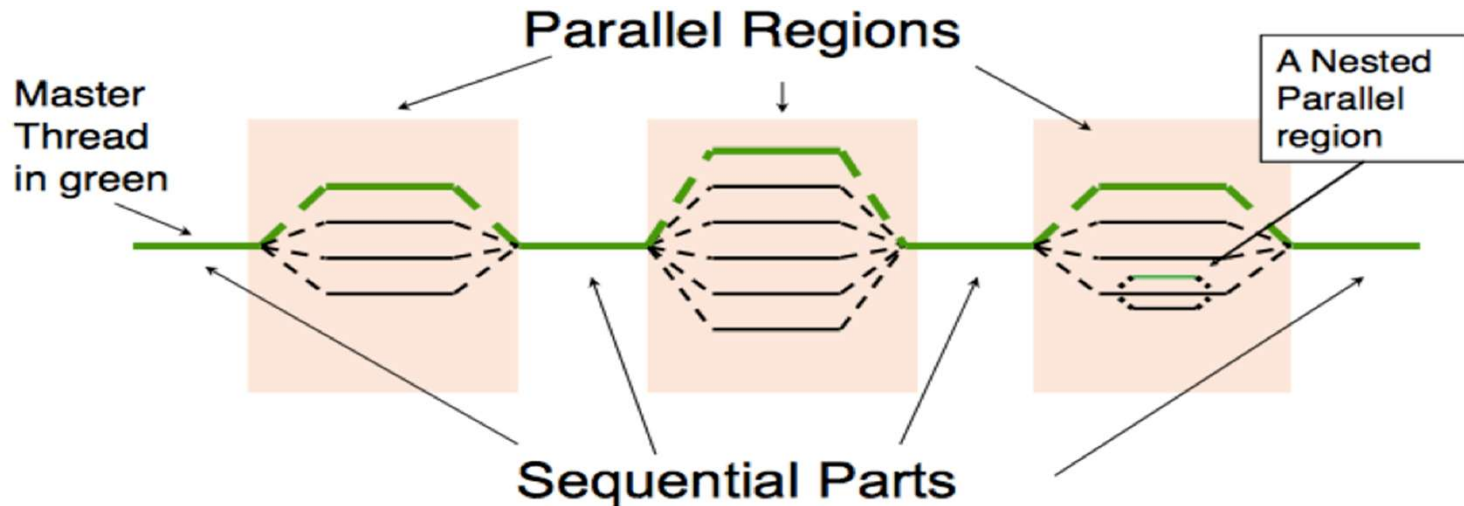
end

}



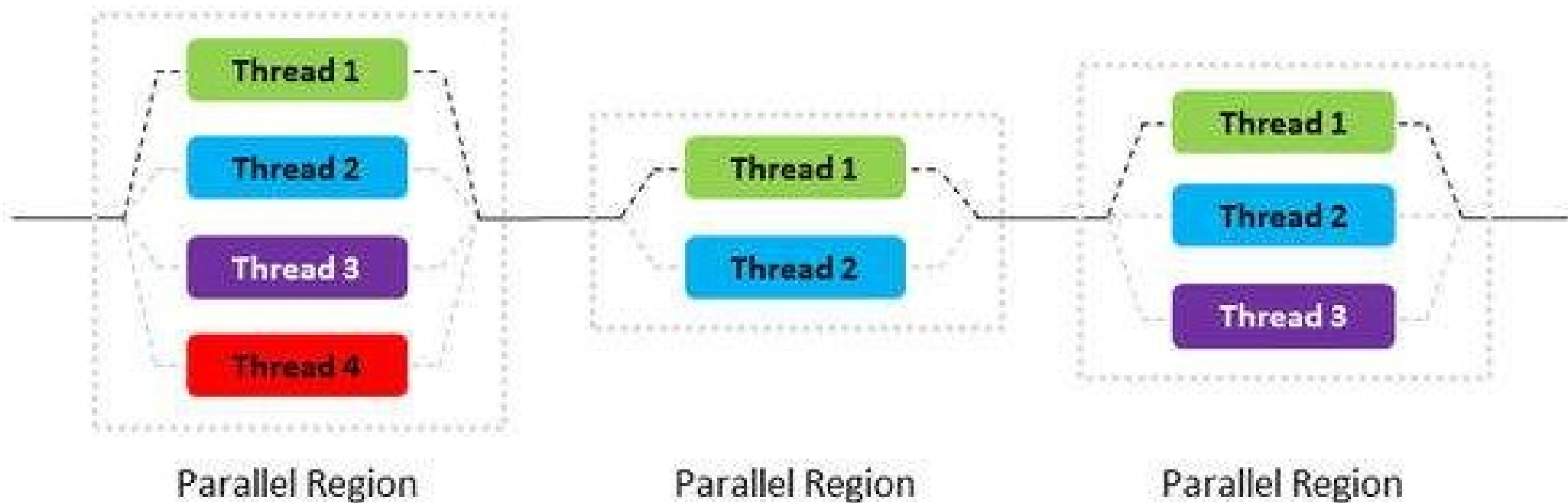
- t_f time stamp at the fork
- t_j time stamp at the joint
- $t_{1,s}$ time stamp of thread 1 being created
- $t_{1,c}$ time stamp of thread 1 completing executing
-  allocated memory for each CPU/thread

Key concepts



- **Advantages:**
-  **Performance Boost:** By using multiple CPU cores, thread-based parallelism can **significantly reduce execution time**.
-  **Efficient Resource Use:** Threads share memory space, making communication between them faster than in process-based parallelism.
-  **Scalability:** As more CPU cores are available (e.g., in modern multi-core processors), thread-based programs can scale better.

Key concepts



Goals of OpenMP



OpenMP (Open Multi-Processing) is a widely used API that supports multi-platform shared memory multiprocessing programming in C, C++, and Fortran. It is designed with specific goals in mind to make parallel programming more accessible and efficient.

- Provide easy-to-use parallel programming support
- Ensure code portability and scalability
- Achieve performance close to hand-optimized parallel code

Goals of OpenMP



Summary Table:

Goal	Description
Easy-to-Use	Simple directives make it easy to write and understand parallel code
Portable and Scalable	Works across platforms and scales with hardware
High Performance	Offers performance close to low-level, hand-tuned parallel implementations

Supported Platforms:

- Linux,
- Windows,
- macOS

Compilers Supporting OpenMP:

- GCC (GNU Compiler Collection)
- Intel C/C++ Compiler
- Clang
- MS Visual C++



1. Compiler Directives (Pragmas)

- Special instructions (e.g., `#pragma omp parallel`) inserted into code.
- Tell the compiler how to parallelize specific sections.
- Control parallel regions, loops, data sharing, synchronization, etc.

2. Runtime Library Routines

- Functions (e.g., `omp_get_thread_num()`, `omp_set_num_threads()`) provided by OpenMP.
- Used to control and query the parallel environment during program execution.

3. Environment Variables

- Set outside the program to influence its parallel behavior.
- Examples:
 - `OMP_NUM_THREADS`: sets number of threads.
 - `OMP_SCHEDULE`: controls loop scheduling.

OpenMP Program Structure

```
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        printf("Hello from thread %d\n", omp_get_thread_num());
    }
}
```

General code layout:

- Include header <omp.h>
- Use #pragma omp directives

Core Syntax and Structured Blocks

```
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        printf("Hello from thread %d\n", omp_get_thread_num());
    }
}
```

#pragma omp parallel – start of a parallel region

#pragma omp for – parallel for-loop

#pragma omp sections – divide tasks among threads

#pragma omp single – execute block by only one thread

#pragma omp critical – protect shared data

Example: Parallel Loop with Structured Block

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
int main() {  
    int i;
```

```
    // Structured block starts here
```

```
    #pragma omp parallel for
```

```
    for (i = 0; i < 5; i++) {
```

```
        printf("Thread %d is processing iteration  
%d\n", omp_get_thread_num(), i);  
    }
```

```
    // Structured block ends after the for loop
```

```
    return 0;
```

```
}
```

Assignment Questions on OpenMP

- Explain the Fork-Join model in OpenMP.
- How does it help in parallel programming?
- Write a simple OpenMP program in C/C++ that demonstrates the use of `#pragma omp parallel` to create multiple threads and print their thread IDs.
- Discuss the roles of OpenMP directives, runtime routines, and environment variables in controlling parallel execution. Give examples.
- How can you specify the number of threads to be used in an OpenMP program?
- Demonstrate with sample code and environment variable settings.

Compiling OpenMP Programs

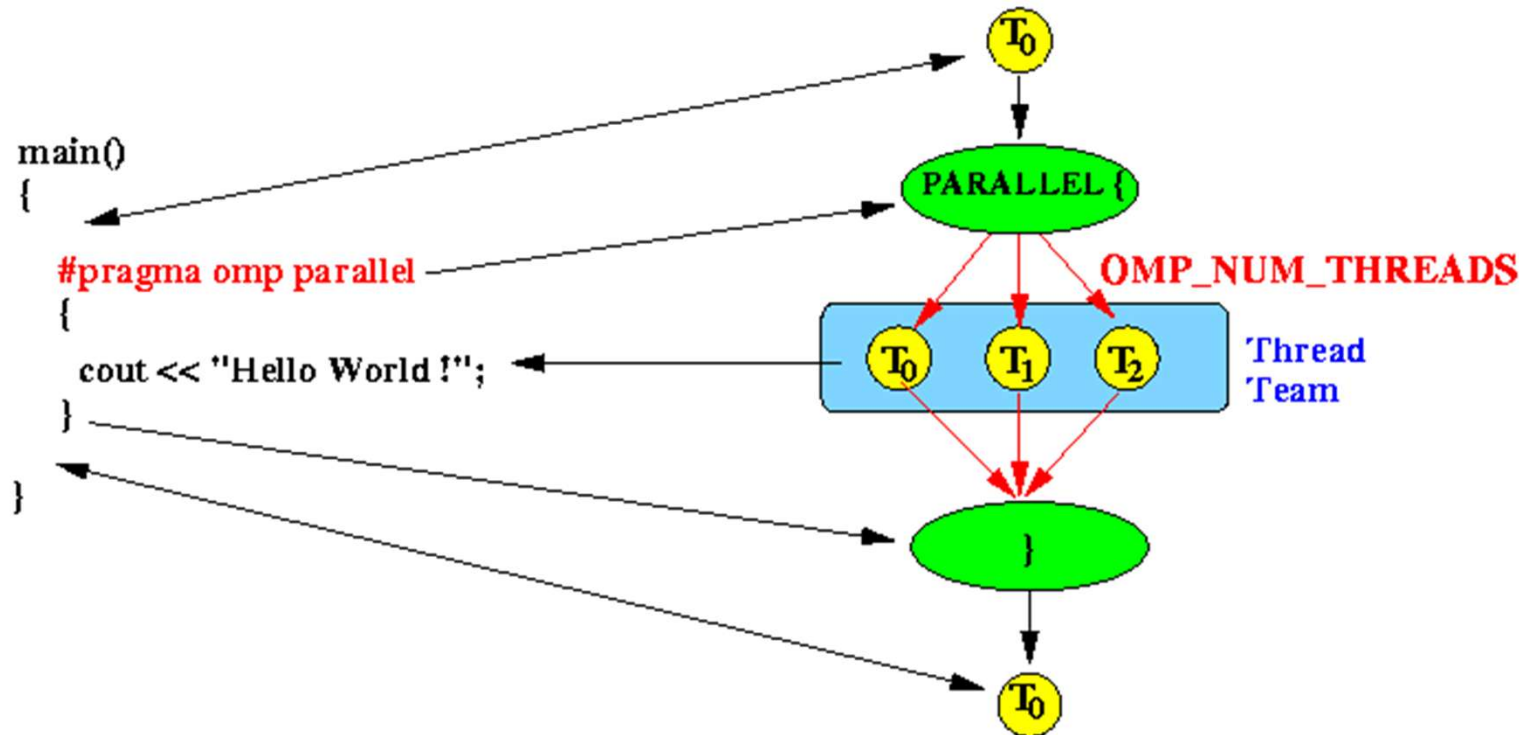
OpenMP (Open Multi-Processing) is an API that supports multi-platform shared memory multiprocessing programming in C, C++, and Fortran. To use OpenMP features, the program must be compiled with specific compiler flags, and thread support can be verified using built-in OpenMP functions.

Compile using OpenMP flags:

- GCC: `gcc -fopenmp program.c -o output`
- Intel: `icc -qopenmp`

Check thread support using `omp_get_max_threads()`

Compiling OpenMP Programs

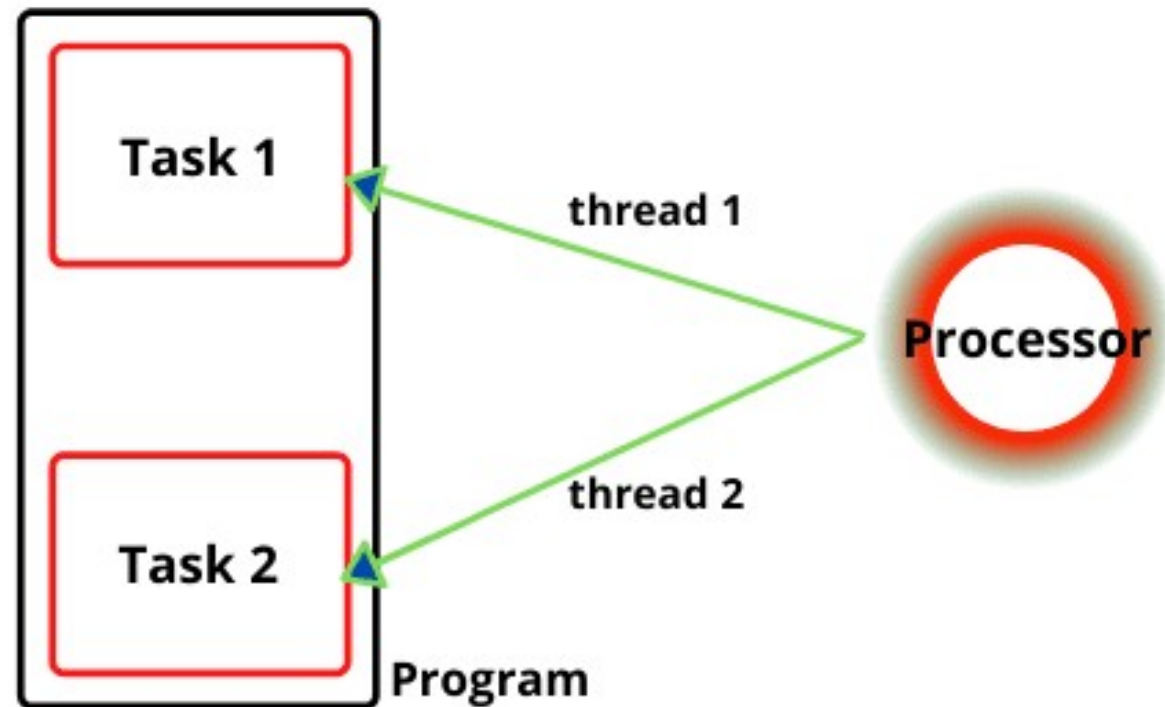


Creating and Managing Threads

In OpenMP, threads are created automatically when a program enters a parallel region. These threads execute concurrently, allowing efficient use of multiple CPU cores for parallel computing.

- Threads created automatically in parallel regions
- Default number of threads = number of CPU cores
- Functions:
 - `omp_get_thread_num()`
 - `omp_get_num_threads()`
- Nested parallelism (disabled by default)

Creating and Managing Threads



Creating and Managing Threads

`omp_get_thread_num()`

- Purpose: Returns the unique thread ID of the current thread.
- Usage: Used inside a parallel region to identify which thread is executing a specific block of code.
- Thread IDs: Start from 0 up to N-1, where N is the total number of threads.

Example:

```
#pragma omp parallel
{
    int id = omp_get_thread_num();
    printf("Thread %d is running\n", id);
}
```

Creating and Managing Threads

`omp_get_num_threads()`

- Purpose: Returns the total number of threads currently running in the parallel region.
- Usage: Useful when you need to know the size of the thread team.
- Note: Should be used inside a parallel region, or it will return 1 (the main thread).

Example:

```
#pragma omp parallel
{
int total = omp_get_num_threads();
printf("Total threads: %d\n", total);
}
```

Specifying Number of Threads

In OpenMP, controlling the number of threads is essential for optimizing parallel execution.

OpenMP provides multiple ways to specify how many threads should be used in a parallel region, allowing developers to manage performance and resource utilization effectively.

- Methods to control thread count:
 - Programmatically: `omp_set_num_threads(n)`
 - Environment Variable: `OMP_NUM_THREADS`
 - Clause: `#pragma omp parallel num_threads(n)`

Specifying Number of Threads

Programmatically: `omp_set_num_threads(n)`

Description: Sets the number of threads to be used in the next parallel region.

Where to use: Call this before entering a `#pragma omp parallel` region.

Example:

```
omp_set_num_threads(4);  
  
#pragma omp parallel{  
  
    // This block will run with 4 threads  
  
}
```

Specifying Number of Threads

Environment Variable: `OMP_NUM_THREADS`

Description: Set this variable in the terminal or shell to define the default number of threads for all parallel regions.

Persistent across runs if set in shell config (like `.bashrc`).

Example (in terminal):

```
export OMP_NUM_THREADS=4
```

```
./a.out
```

Specifying Number of Threads

Clause in Directive:

```
#pragma omp parallel num_threads(n)
```

Description: Directly specifies the number of threads for a specific parallel region.

Overrides any previously set value by `omp_set_num_threads()` or `OMP_NUM_THREADS`.

Example:

```
#pragma omp parallel num_threads(4){  
// Executes with exactly 4 threads  
}
```


Thread Management and IDs

In OpenMP, identifying threads is essential for managing their behavior and coordination within parallel regions. Each thread is assigned a unique ID, which can be used for conditional execution or specialized tasks.

- Thread identification:
 - `omp_get_thread_num()` – returns threadID
 - `omp_get_num_threads()` – returns total threads
- Master thread has ID = 0
- Use thread ID for conditional execution

OpenMP is widely used to accelerate computational tasks by leveraging multi-core processors. Its parallel programming capabilities make it ideal for performance-critical applications across various scientific, engineering, and real-time domains.

- Scientific Simulations
- Image & Signal Processing
- Machine Learning Models (Preprocessing, Training loops)
- Financial Modelling
- Game Physics Engines

Assignment Questions on OpenMP

- Explain thread management in OpenMP. How are threads created, managed, and destroyed during the execution of a parallel region? What are environment variables in OpenMP? Explain the role of `OMP_NUM_THREADS` and `OMP_SCHEDULE`.
- Explain how you would compile and run an OpenMP program on a Linux system using GCC compiler. Provide the compilation command and a brief explanation.

Assignment Questions on OpenMP

- Explain the structure of an OpenMP program with emphasis on the parallel region construct.
- Describe the applications where OpenMP can be effectively used.
- What are the limitations of OpenMP compared to other parallel programming models like MPI?
- Write an OpenMP program to parallelize a for-loop that computes the sum of an array of integers. Discuss the potential issues and how OpenMP addresses them.

Applications of OpenMP

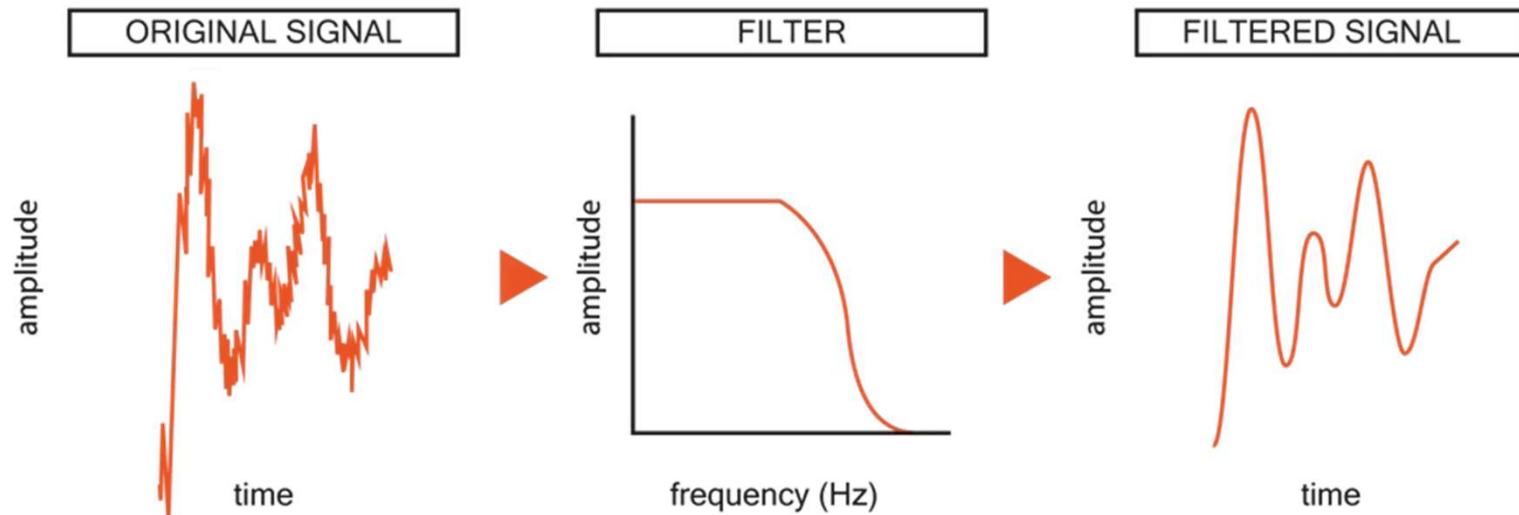


1. Scientific Simulations

Use Case: Weather prediction, climate modeling, molecular dynamics, astrophysics.

Why OpenMP?: These simulations involve massive numerical computations on large datasets. OpenMP helps by parallelizing loops and computations across multiple CPU cores, speeding up the simulation significantly.

Applications of OpenMP

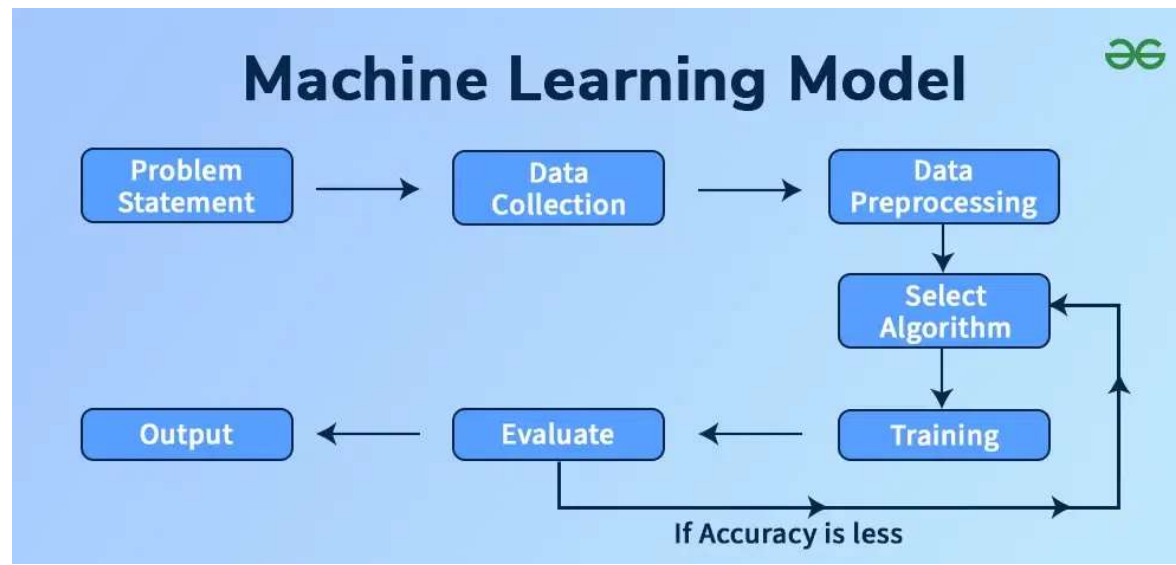


2. Image & Signal Processing

Use Case: Image filtering, edge detection, Fourier transforms, noise reduction.

Why OpenMP?: Processing each pixel or signal segment can be done independently, making them ideal for parallelization. OpenMP improves processing speed in real-time applications like medical imaging or audio processing.

Applications of OpenMP



3. Machine Learning Models (Preprocessing, Training Loops)

Use Case: Data normalization, feature extraction, model training (especially for CPU-based libraries).

Why OpenMP?: ML workflows often involve repetitive, data-parallel tasks. OpenMP allows parallel preprocessing and faster CPU-based model training loops in frameworks like Scikit-learn or TensorFlow (when not using GPU).

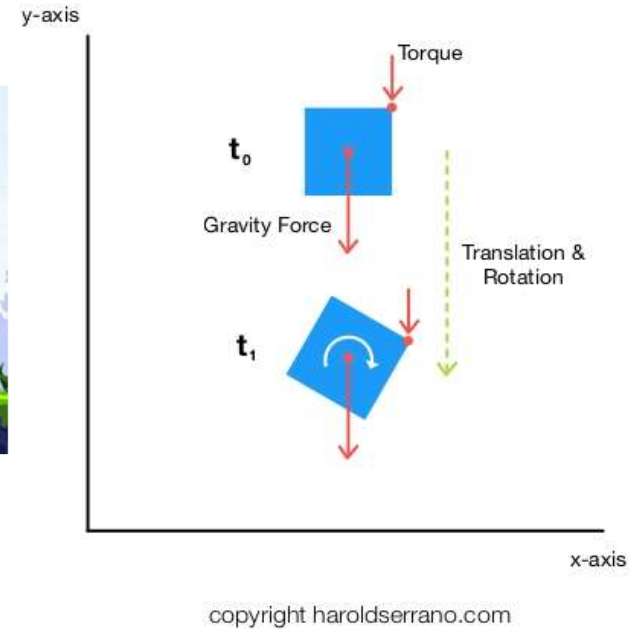
Applications of OpenMP



4. Financial Modelling

Use Case: Risk analysis, Monte Carlo simulations, option pricing models.

Why OpenMP?: These computations involve heavy mathematical operations that can be run concurrently. OpenMP enables fast parallel execution of thousands of simulations.



5. Game Physics Engines

Use Case: Collision detection, fluid dynamics, particle systems, character animation.

Why OpenMP?: Game engines use real-time calculations where parallel threads can handle different physics components simultaneously, reducing CPU load and enhancing frame rates.

1.What is the primary programming model used by OpenMP?

A) Message Passing Interface (MPI)

B) Fork-Join Model

C) Dataflow Model

D) Actor Model

2.Which of the following is NOT a component of the OpenMP API?

A) Directives

B) Runtime routines

C) Environment variables

D) Garbage collector

3.What is the default behavior of an OpenMP parallel region regarding the number of threads?

- A) One thread only
- B) Maximum number of available processors
- C) Number specified by the user or system environment**
- D) Number of processes running on the system

4.Which directive is used to specify a parallel region in OpenMP?

- A) #pragma omp parallel**
- B) #pragma omp for
- C) #pragma omp section
- D) #pragma omp critical

5 How do you specify the number of threads in an OpenMP program?

- A) omp_set_num_threads() function
- B) Setting environment variable OMP_NUM_THREADS
- C) Both A and B**
- D) Using a command line argument

6 Which of the following is true about thread IDs in OpenMP?

- A) All threads have the same thread ID
- B) Thread IDs are unique integers starting from 0**
- C) Thread IDs are assigned randomly
- D) Thread IDs are always even numbers

7.What happens at the end of an OpenMP parallel region?

- A) Threads continue running indefinitely
- B) Threads are joined back into the master thread**
- C) Threads are terminated permanently
- D) Threads are duplicated

8.Which OpenMP directive is used for specifying work-sharing constructs like loops?

- A) parallel
- B) for**
- C) section
- D) single

9. Which platforms support OpenMP?

- A) Windows only
- B) Linux only
- C) UNIX, Linux, and Windows**
- D) Embedded systems only

10. Which of the following is NOT a goal of OpenMP?

- A) Ease of programming
- B) Portability
- C) Scalability across multiple nodes in a cluster**
- D) Performance

Summary

- OpenMP is ideal for shared-memory parallelism
- Simple directives allow easy integration with existing code
- Flexibility in managing threads, workload, and data
- Widely supported and efficient for multicore programming

Recommended YouTube Tutorials on OpenMP

1. “Tim Mattson – Introduction to OpenMP” (Intel)

An accessible series by one of OpenMP’s core contributors, covering basics and core constructs.

2. “OpenMP Tutorial: Hello World & Parallel For Loops” (LLNL)

Covers fundamental usage, compilation flags, and loop parallelization in C/C++.

3. “OpenMP In C++ – Fast Guide”

A step-by-step demo of `#pragma omp parallel for`, thread IDs, and scheduling.

4. “OpenMP Task Parallelism Deep Dive”

Explains more advanced features like task constructs, synchronization, and reduction.

5. “Hands-on OpenMP – Parallel Sum & Matrix Multiply”

Builds a parallel reduction and matrix multiplication example, ideal for practical programming.

References

S.No	Book Title	Author
1	The Sourcebook of Parallel Computing	Jack Dongarra, Geoffrey Fox, and Ken Kennedy
2	Petascale Computing: Algorithms and Applications	David A. Bader (Ed.)
3	Parallel and High Performance Computing	Robert Robey and Yuliana Zamora
4	High Performance Computing: Modern Systems and Practices	Thomas Sterling, Maciej Brodowicz, and Matthew Anderson

Reference Books:

S.No	Book Title	Author
1	High Performance Computing: Modern Systems and Practices	Thomas Sterling, Matthew Anderson, and Maciej Brodowicz
2	Using OpenMP: Portable Shared Memory Parallel Programming	Barbara Chapman, Gabriele Jost, and Ruud van der Pas
3	Introduction to Parallel Computing	Ananth Grama, Anshul Gupta, George Karypis, and Vipin Kumar
4	The OpenMP Common Core: A Hands-on Exploration	Tim Mattson et al.
5	A Hands-on Introduction to OpenMP	Tim Mattson and Larry Meadows
6	OpenMP Tasking Explained	Ruud van der Pas
7	An Introduction to Parallel Programming	Peter Pacheco

Thank You